

Learn Home Automation with Arduino

Lesson 10: Final Project: Automated Room

Final Project: Automated Room – Study Notes

Key Concepts

1. **Integration of Sensors & Actuators** – Using temperature, humidity, and light sensors together with relays to control lights, fans, and HVAC elements.
2. **Wi-Fi Connectivity** – Sending sensor data to the cloud / smartphone and receiving remote commands via MQTT, HTTP, or a dedicated app.
3. **Voice Control** – Linking the room to Alexa, Google Assistant, or a custom voice recognition service to turn devices on/off with spoken commands.
4. **Power Management & Safety** – Properly sizing relays, using opto isolators, and adding protective components (flyback diodes, fuses).
5. **Programming Logic** – Writing firmware (Arduino/ESP8266/NodeMCU) that reads sensors, decides when to actuate, and handles network/voice events.

Summary

This lesson walks you through building a **smart, automated room** that can autonomously adjust lighting and climate while also responding to remote and voice commands. You'll learn how to wire temperature, humidity, and ambient light sensors to a microcontroller (e.g., ESP32 or ESP8266), read their values, and use those readings to drive relays that switch lights, ceiling fans, or a heater.

Next, you'll connect the microcontroller to your Wi-Fi network, publish sensor data to a cloud broker (MQTT) or a simple REST endpoint, and subscribe to control topics. With that link in place, you can create a mobile dashboard or integrate with a voice assistant platform so that saying "Hey Google, set the room to comfortable" triggers the appropriate relay actions. The project emphasizes safe power handling, modular code organization, and testing each subsystem before full integration.

Important Points

- **Sensor choices**
 - DHT22 or BME280 for temperature & humidity.
 - BH1750 or photoresistor for ambient light.
- **Relay selection**
 - Use a 5V/3.3V logic level relay module with built-in opto isolator.
 - Add a flyback diode (e.g., 1N4007) across inductive loads if the module lacks one.
- **Power considerations**
 - Keep the microcontroller on a stable 3.3V/5V supply separate from the high-voltage AC load.
 - Include a fuse (250mA–500mA) on the AC side for safety.
- **Wi-Fi & Cloud**

- ESP Home or Arduino IDE can handle MQTT (`client.publish()`, `client.subscribe()`).
 - Use **TLS** or at least a username/password for broker authentication.
 - **Voice integration**
 - For Alexa: create a **Smart Home Skill** and expose an HTTPS endpoint that forwards intents to your MQTT broker.
 - For Google Assistant: use **Dialogflow** or the **Google Smart Home API** similarly.
 - **Software flow**
1. Read sensors & publish JSON payload.
 2. Evaluate thresholds (e.g., `if (lux < 200) turn on lights`).
 3. Listen for inbound MQTT/HTTP commands & trigger relays.
 4. Send status updates back to the cloud.
- **Testing strategy**
- Verify each sensor reading in isolation.
 - Test relay switching with a low voltage lamp before connecting mains.
 - Simulate network loss and ensure the system fails safe (e.g., turn off HVAC).

Examples

Example 1 – Automatic Lighting

```

`cpp
// ESP8266 pseudo code
#include <ESP8266WiFi.h>
#include <PubSubClient.h>
#include <BH1750.h>
BH1750 lightMeter;
WiFiClient espClient;
PubSubClient client(espClient);
const int RELAY_PIN = D5;
const uint16_t LUX_THRESHOLD = 150; // dim room !' lights on
void setup() {
  pinMode(RELAY_PIN, OUTPUT);
  digitalWrite(RELAY_PIN, HIGH); // relay off (active low)
  Wire.begin();
  lightMeter.begin();
  WiFi.begin("SSID","PASS");
  client.setServer("mqtt.mybroker.com",1883);
}
void loop() {
  if (!client.connected()) reconnect();
  client.loop();
  uint16_t lux = lightMeter.readLightLevel();
  if (lux < LUX_THRESHOLD) digitalWrite(RELAY_PIN, LOW); // turn lights on

```

```

else digitalWrite(RELAY_PIN, HIGH);           // turn off
// publish reading
char msg[50];
sprintf(msg,50,"{\"lux\":%d}",lux);
client.publish("room1/light",msg);
delay(5000);
}
...

```

***Explanation:** The BH1750 measures ambient light. If it drops below 150 /lux, the relay (active low) closes, powering a ceiling LED strip. The current lux value is published to MQTT for monitoring.

Example 2 – Voice Controlled Climate

1. ****MQTT topic**:** `room1/climate/set` receives JSON like `{"mode":"cool","temp":24}`.
2. ****Alexa Smart Home Skill**** maps the intent “Set thermostat to 24 degrees” to an HTTPS POST that forwards the JSON to the broker.
3. Firmware snippet:

```

`cpp
void callback(char* topic, byte* payload, unsigned int length) {
    String msg;
    for (unsigned int i=0;i<length;i++) msg += (char)payload[i];
    DynamicJsonDocument doc(200);
    deserializeJson(doc, msg);
    String mode = doc["mode"]; // "cool" or "heat"
    int target = doc["temp"]; // desired temperature
    if (mode == "cool") digitalWrite(FAN_RELAY, LOW);
    else if (mode == "heat") digitalWrite(HEATER_RELAY, LOW);
    // simple example – real project would include PID control
}
...

```

***Explanation:** When Alexa sends the command, the ESP receives the JSON via MQTT, parses it, and activates the appropriate relay (fan for cooling, heater for heating). The system can be expanded with a PID loop that modulates fan speed using PWM.

Quick Review

1. ****What safety component protects a relay's coil from inductive kickback, and why is it needed?****
2. ****Name two cloud protocols you can use to exchange sensor data and control commands.****
3. ****How does an active low relay differ from an active high relay in wiring?****
4. ****What JSON payload would you publish to report a temperature of 22 /°C and humidity of 55 /%?****
5. ****Describe one advantage of handling voice commands through an MQTT broker rather than direct HTTP calls.****

Further Reading

- ****ESP Home & Home Assistant**** – building YAML based automations without writing code.
- ****MQTT Security**** – TLS, certificates, and token based authentication.
- ****PID Temperature Control**** – implementing smooth climate regulation.
- ****Power Electronics for Hobbyists**** – deeper dive into relays, MOSFETs, and solid state switches.
- ****Voice Assistant SDKs**** – Alexa Smart Home Skill Kit, Google Smart Home Action, and open source alternatives like Mycroft.

